RAPPORT TECHNIQUE

Test Technique Laravel

Application de gestion de réservations immobilières

avec authentification Breeze · Livewire · Filament

BAH Ibrahima Kalil

Candidat au poste de Stagiaire Développeur Web

Étudiant passionné par le développement web, je soumets ce projet dans le cadre du test technique pour un stage chez InnovQube. Ce rapport documente l'application de réservations immobilières que j'ai développé en utilisant Laravel, Livewire et Filament, démontrant ma capacité à concevoir et livrer une application web complète et structurée.

GitHub `https://github.com/kalilbah/Projet-laravel-booking-app`

Destinataire rh@innovqube.com

Date Mars 2026

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du test technique Laravel proposé par InnovQube. L'objectif était de construire, à partir d'un squelette Laravel vierge, une application complète de gestion de réservations immobilières couvrant les besoins suivants :

- Un espace public permettant de consulter un catalogue de biens
- Un espace utilisateur avec tableau de bord et gestion des réservations
- Un module de réservation dynamique Livewire
- Un panneau d'administration Filament
- Une séparation claire entre les rôles administrateur et client

Le projet a été initialisé avec la commande :

```
composer create-project laravel/laravel laravel-test
```

puis enrichi progressivement pour répondre à l'ensemble des exigences fonctionnelles du cahier des charges.

2. Stack Technique

2.1 Dépendances PHP principales

Package	Version	Rôle
`laravel/framework`	`^13.0`	Framework principal
`laravel/tinker`	`^3.0`	REPL interactif
`livewire/livewire`	`^4.2`	Composants dynamiques
`filament/filament`	`^5.4`	Panneau d'administration
`laravel/breeze`	`^2.4`	Authentification (dev)

2.2 Dépendances front-end

- TailwindCSS + @tailwindcss/forms - stylisation de l'interface
- Vite + laravel-vite-plugin - bundler et compilation des assets
- AlpineJS - réactivité légère côté client
- Axios - requêtes HTTP asynchrones
- Concurrently - lancement parallèle des serveurs de développement

3. Architecture du Projet

3.1 Modèles et relations

Trois modèles principaux ont été créés pour couvrir les besoins métier :

Modèle	Champs principaux	Relations
`User`	`name, email, role,`	hasMany Booking

	`profile_photo_path`	
`Property`	`title, description, address, price_per_night, image`	hasMany Booking
`Booking`	`user_id, property_id, start_date, end_date, status`	belongsTo User, Property

3.2 Gestion des rôles

Deux rôles sont définis dans la colonne role de la table users :

- admin - accès au panneau Filament sur /admin
- customer - accès à l'espace utilisateur

La redirection après connexion est gérée automatiquement par le contrôleur AuthenticatedSessionController selon le rôle détecté. Les routes utilisateur sont protégées par le middleware personnalisé EnsureCustomer, qui empêche un administrateur d'accéder à l'espace client.

3.3 Migrations

Les migrations suivantes ont été créées pour le projet :

- create_properties_table - titre, description, adresse, prix par nuit, image
- create_bookings_table - clés étrangères user_id et property_id, dates, statut
- add_role_to_users_table - champ role avec valeur par défaut customer
- add_profile_photo_path_to_users_table - champ pour la photo de profil

4. Fonctionnalités Réalisées

4.1 Espace public

L'espace public est accessible sans authentification et comprend :

- Page d'accueil personnalisée avec présentation du catalogue de biens
- Liste paginée des propriétés disponibles
- Fiche détaillée pour chaque bien (titre, description, adresse, prix)
- Navigation adaptée selon le statut de connexion de l'utilisateur

4.2 Authentification (Laravel Breeze)

L'authentification est gérée via Laravel Breeze avec :

- Inscription avec confirmation de mot de passe
- Connexion avec redirection intelligente selon le rôle
- Déconnexion sécurisée avec destruction de session
- Protection CSRF sur tous les formulaires

4.3 Module de réservation Livewire

Le composant BookingManager, développé avec Livewire 4, permet à un utilisateur connecté de réserver un bien depuis sa fiche détaillée. Les fonctionnalités incluent :

- Sélection des dates d'arrivée et de départ via des champs date
- Calcul dynamique du nombre de nuits et du montant total estimé
- Validation serveur des dates (cohérence, disponibilité)
- Enregistrement de la réservation avec statut pending
- Blocage des visiteurs non connectés (redirection vers la connexion)
- Blocage des administrateurs (accès réservé aux clients)

Le composant utilise les directives wire:model pour la liaison bidirectionnelle des données et wire:click pour les actions.

4.4 Espace utilisateur

Accessible uniquement aux comptes de type customer, le tableau de bord utilisateur permet de :

- Visualiser l'ensemble des réservations du compte connecté
- Consulter le détail de chaque réservation (bien, dates, statut)
- Annuler une réservation via un bouton dédié
- Accéder rapidement au catalogue de biens
- Gérer son profil (informations personnelles et photo)

4.5 Gestion du profil utilisateur

La page de profil permet à l'utilisateur de :

- Modifier son nom et son adresse e-mail
- Changer son mot de passe
- Uploader une photo de profil (stockée dans storage/app/public/profile-photos)
- Supprimer sa photo existante
- Afficher un avatar par défaut si aucune photo n'est définie

La photo de profil est affichée dans la barre de navigation et sur la page profil.

4.6 Panneau d'administration Filament

Le panneau Filament, disponible sur /admin, offre une interface complète d'administration :

Section	Fonctionnalités
Propriétés	Lister, créer, modifier et supprimer les biens immobiliers
Réservations	Consulter toutes les réservations, créer manuellement avec attribution par email
Clients	Consulter et éditer les comptes utilisateurs de type customer
Administrateurs	Lister les comptes admin en lecture seule
Tableau de bord	Indicateurs globaux et réservations récentes

5. Guide d'Installation

5.1 Prérequis

- PHP >= 8.2
- Composer >= 2.x
- Node.js >= 18.x et npm
- Base de données MySQL

5.2 Étapes d'installation

Étape 1 - Cloner le dépôt et installer les dépendances

```
composer install
npm install
```

Étape 2 - Configurer l'environnement

```
cp .env.example .env
```

Configurer DB_DATABASE, DB_USERNAME, DB_PASSWORD dans .env, puis :

```
php artisan key:generate
```

Étape 3 - Base de données et données de démonstration

```
php artisan migrate:fresh -seed
```

Étape 4 - Lien de stockage public

```
php artisan storage:link
```

Étape 5 - Lancer le projet

```
composer run dev
```

Ou manuellement :

```
php artisan serve
npm run dev
```

6. Comptes de Démonstration

6.1 Compte Administrateur

Email	admin@example.com
Mot de passe	password
URL d'accès	http://127.0.0.1:8000/admin/login

6.2 Compte Client

Email	test@example.com
Mot de passe	password
URL d'accès	http://127.0.0.1:8000/login

7. Principaux Fichiers Modifiés ou Ajoutés

7.1 Backend

- app/Models/User.php - modèle utilisateur étendu (rôle, photo, relations)
- app/Http/Controllers/ProfileController.php - gestion du profil et de la photo
- app/Http/Controllers/Auth/AuthenticatedSessionController.php - redirection selon rôle
- app/Http/Controllers/UserBookingController.php - annulation des réservations
- app/Http/Middleware/EnsureCustomer.php - protection des routes client
- app/Livewire/BookingManager.php - logique du composant de réservation
- app/Providers/Filament/AdminPanelProvider.php - configuration du panneau admin
- app/Filament/Resources/UserResource.php - gestion des clients
- app/Filament/Resources/AdminResource.php - liste des administrateurs (lecture seule)
- app/Filament/Resources/BookingResource.php - gestion des réservations

7.2 Routes et migrations

- routes/web.php - définition de toutes les routes publiques et protégées
- database/factories/PropertyFactory.php - génération de biens fictifs
- database/seeders/DatabaseSeeder.php - alimentation de la base avec des données de démo
- database/migrations/ - 4 migrations personnalisées ajoutées

7.3 Vues Blade

- resources/views/welcome.blade.php - page d'accueil
- resources/views/layouts/navigation.blade.php - navigation avec photo de profil
- resources/views/dashboard.blade.php - tableau de bord utilisateur
- resources/views/properties/show.blade.php - fiche détaillée du bien
- resources/views/profile/edit.blade.php - page de gestion du profil
- resources/views/livewire/booking-manager.blade.php - vue du composant Livewire

8. Tests

Le projet inclut la configuration PHPUnit standard de Laravel. Pour lancer la suite de tests :

```
php artisan test
```

La suite couvre les tests unitaires et fonctionnels fournis par Laravel Breeze et les tests personnalisés ajoutés au projet.

9. Synthèse

Ce projet répond à l'ensemble des critères du test technique :

Critère	Statut
Projet Laravel fonctionnel avec authentification Breeze	✅ **Réalisé**
Gestion des propriétés et réservations	✅ **Réalisé**
Interface Blade avec TailwindCSS	✅ **Réalisé**
Composant Livewire pour la réservation	✅ **Réalisé**
Panneau Filament pour la gestion des données	✅ **Réalisé**
Séparation des rôles admin / client	✅ **Réalisé**
Gestion de la photo de profil	✅ **Réalisé (bonus)**
Données de démonstration via seeders	✅ **Réalisé (bonus)**

Ce test technique a permis de démontrer la mise en œuvre d'une application Laravel moderne, structurée et fonctionnelle, couvrant l'ensemble du spectre demandé : authentification, modèles relationnels, composants réactifs Livewire, administration Filament et personnalisation de l'interface utilisateur.